

Talent Intelligence System Design

04/06/2026 09:04:13

Act as a system design architect Analyze this problem and produce: 1. Problem...
Here's a structured system design breakdown for the Talent Intelligence & Ranking...
can you explain below terms in simple language and with example Ranking request...
Here's a simple, clear explanation of each term + your added questions. 1) Rankin...
add the above questions to relevant section under "Open Questions"
Here's your updated Open Questions section with your additional points properly...
based on the above design a high-level architecture initially for MVP and later on c...
Here's a practical, MVP-first high-level architecture using .NET + Azure, designed s...
for MVP at initial level thinking of below, validate the pros and cons for it keeping t...
Your approach is actually very solid for an MVP, especially given the 4-hour...
more information on cosmosDB indexing part and simple example explanation of...
Great question — this is exactly where most MVP designs either succeed or silently...
here pdf will be scanned and the whole text can be stored now how Skills can be...
Short answer: Yes, it adds some complexity — but you can keep it very manageabl...
It violates the actual problem statement of keyword searching instead of semantic...
You're asking the right architectural question — this is exactly where many solution...
Cosmos vs Azure Cognitive Search (vector-first design)
Great comparison — this is a key architectural decision and directly impacts how "A...
let's choose only cosmosDB to keep and later on add Cognitive Search for...
Great — this is a clean, practical decision. You're keeping MVP simple with Cosmos...
now the infra part and implementation is clear summarize based on the above...
Here's a clean, interview-ready summary of your design based on everything we...
yes design.md and include below and add if anything is missing - Architecture - Da...
Here's a clean, submission-ready DESIGN.md you can directly use. 📄 DESIGN.md...
for implementing the solution architecture is clean architecture a best fit ?
Short answer: Yes — but only a lightweight version of Clean Architecture is the bes...
Generate actual .NET project structure (folders + files)

Here's a practical .NET project structure using a lightweight Clean Architecture,...
in the above Design.md for a Talent Intelligence & Ranking Engine (MVP) built on...
Here is your updated, submission-ready DESIGN.md with all requested changes...



Act as a system design architect

Analyze this problem and produce:

1. Problem understanding
2. Assumptions
3. Functional requirements
4. Non-functional requirements
5. Open questions

Scenario: Talent Intelligence & Ranking Engine

You have been asked to rebuild a legacy recruitment system for a hiring platform operating at scale. The current keyword-matching approach produces low-quality candidate-to-role matches, and the business needs a smarter, AI-native solution.

Context & Scale

Dimension

Details

Active Candidates

50,000 profiles in the system

Live Job Openings

1,500 roles open at any time

Current Problem

Keyword matching — produces poor-quality matches

Goal

A ranked "Top 10 Candidates" list per job role

What You Need to Build

Design and implement a service that:

Autonomously parses resumes and job descriptions from disparate file formats

Builds a pipeline from document ingestion through to a ranked candidate output

Returns the top 10 best-matched candidates for any given job role

Handles high-concurrency ingestion without data loss or

bottlenecks

Technical Constraints

Your solution must address the following:

1. File Format Handling:

Ingestion must support multiple formats (PDF, DOCX, etc.)

2. Data Pipeline

Define the full flow: from raw document intake → parsing → scoring → ranked output

3. Cold Start Accuracy

Explain how the system ensures quality matches for a brand-new job posting that has no historical data or prior matches to learn from

Deliverables

Submit the following when you are done:

Deliverable

Description

Repository / Code

A working "thin-slice" implementation covering the core logic of your solution

DESIGN.md

Architecture document covering: system design, data flow, scaling approach, and known failure modes

AI Interaction Log

Export your chat history from Cursor, Windsurf, or ChatGPT, whichever tool you used during the build

Tools & Environment

You are encouraged to use an AI-native development environment. Recommended options:

Cursor

Windsurf

VS Code with GitHub Copilot or Aider

ChatGPT (if using this, export your full conversation when submitting)

Submission Checklist

Code repository (or zip of your project)

DESIGN.md with architecture details

AI chat / prompt history export



Here's a structured system design breakdown for the **Talent Intelligence & Ranking Engine**.

1) Problem Understanding

We need to replace a **naive keyword-matching recruitment system** with an **AI-native ranking engine** that:

- Ingests resumes + job descriptions from multiple formats (PDF, DOCX, etc.)
- Extracts structured + semantic meaning
- Matches candidates to job roles intelligently
- Produces a **Top 10 ranked candidates per job**
- Works at moderate scale:
 - 50K candidates
 - 1.5K active jobs
- Handles **concurrent ingestion + processing**
- Maintains **high-quality results even for new jobs (cold start)**

👉 Core challenge:

Move from **lexical matching** → **semantic + contextual matching + ranking**

2) Assumptions

Data Assumptions

- Resumes and JDs are mostly text-based (not scanned images, or OCR fallback exists)
- Candidates update resumes infrequently
- Jobs change more frequently than candidates
- Profiles can be enriched over time

Traffic Assumptions

- Read-heavy system (matching queries)
- Burst ingestion during hiring drives
- Ranking requests: ~100–500 QPS peak



ML/AI Assumptions

- Pretrained models available (e.g., embeddings, NER)
- No labeled dataset initially → must rely on zero-shot / heuristic ranking
- Feedback loop (clicks, hires) available later

System Assumptions

- Event-driven architecture preferred
 - Eventually consistent pipeline acceptable
 - Slight latency acceptable (e.g., 1–3 seconds per query)
-

3) Functional Requirements

A. Document Ingestion

- Upload resumes (PDF, DOCX)
- Upload job descriptions
- Parse and normalize text

B. Data Processing Pipeline

- Extract structured data:
 - Skills
 - Experience
 - Education
 - Titles
- Generate embeddings (semantic vectors)

C. Matching Engine

- Match candidates to job descriptions
- Compute relevance score

D. Ranking Service

- Return **Top 10 candidates per job**
- Allow filtering (optional future):
 - Location
 - Experience range
 - Salary band

E. Search & Retrieval

- Fast retrieval of candidate vectors
- Similarity search

F. Re-ranking Layer

- Combine multiple signals:
 - Semantic similarity
 - Skill overlap
 - Experience match
 - Recency

G. Admin / API Layer

- APIs:
 - Upload resume / JD
 - Get top candidates for job
 - Recompute rankings

4) Non-Functional Requirements

Scalability

- Handle:
 - 50K candidates → grows to millions
 - Parallel ingestion
- Horizontal scaling (stateless services)

Performance

- Ranking latency: < 2 seconds
- Vector search: < 100ms

Reliability

- No data loss during ingestion
- Retry mechanisms
- Idempotent processing

Consistency

- Eventual consistency acceptable for rankings
- Strong consistency for stored profiles

Observability

- Metrics:
 - Parsing success rate
 - Ranking latency
 - Matching quality (CTR, hire rate)
- Logging + tracing

Extensibility

- Add new ranking signals easily
- Plug new ML models without redesign

Security

- PII protection (resumes contain sensitive data)
- Access control

5) Open Questions

Product Questions

- What defines a “good match”? (skills vs experience vs culture)
- Should recruiters tweak ranking weights?
- Do we need explainability? (“Why this candidate?”)

Data Questions

- Do we have labeled hiring outcomes?
- Are resumes multilingual?
- Do we need OCR for scanned PDFs?

ML Questions

- Which embedding model? (OpenAI, SBERT, custom)
- Do we fine-tune or use zero-shot initially?
- How do we handle bias?

System Questions

- Real-time vs batch ranking?
- Precompute matches or compute on-demand?
- Storage choice for embeddings (FAISS, Pinecone, etc.)?

(Bonus) High-Level Architecture

Pipeline Overview

```
[Upload Service]
  ↓
[Document Storage (S3)]
  ↓
[Parsing Service]
  ↓
[Feature Extraction + NLP]
  ↓
```

```
[Embedding Service]
  ↓
[Vector DB]
  ↓
[Matching + Ranking Service]
  ↓
[Top 10 Candidates API]
```

Key Design Choices

1. Parsing Layer

- Use:
 - Apache Tika / custom parsers
 - OCR fallback (Tesseract)

2. Feature Extraction

- NER models for:
 - Skills
 - Organizations
 - Roles
- Rule-based + ML hybrid

3. Embeddings

- Use transformer-based embeddings:
 - Resume → vector
 - JD → vector

4. Vector Search

- Approximate Nearest Neighbor (ANN):
 - FAISS / Pinecone / Weaviate

5. Ranking Formula (Cold Start Friendly)

```
Score =
  w1 * semantic_similarity +
  w2 * skill_overlap +
  w3 * experience_match +
  w4 * recency +
  w5 * location_match
```

👉 No historical data needed → solves cold start

Cold Start Strategy (Critical)

For a **new job with no history**:

1. Use **semantic embeddings** of JD
2. Retrieve top-N candidates via vector similarity
3. Apply heuristic ranking:
 - Skill extraction overlap
 - Years of experience alignment
4. Optional:
 - Use LLM scoring (pairwise comparison)
 - Boost exact skill matches

👉 This ensures **high baseline quality without training data**

Failure Modes & Mitigations

Failure	Mitigation
Bad parsing	Retry + fallback parser
OCR failure	Confidence scoring
Embedding drift	Version embeddings
Slow ranking	Cache top results

Failure	Mitigation
Skewed results	Diversity constraints

Thin-Slice Implementation Suggestion

For your submission:

Minimal Working Version

- Resume/JD ingestion (PDF → text)
- Use pretrained embedding model
- Store vectors (FAISS local)
- Compute cosine similarity
- Return top 10 candidates

Stack

- Python (FastAPI)
- FAISS (vector search)
- SQLite / Postgres (metadata)
- HuggingFace / OpenAI embeddings

What Your DESIGN.md Should Include

- Architecture diagram
- Data flow
- API contracts
- Ranking logic
- Scaling strategy
- Tradeoffs
- Failure handling

If you want next step, I can:

- Generate a **complete code repo structure**
- Write **DESIGN.md**
- Provide **working FastAPI + FAISS implementation**
- Or simulate **interview deep dive (like system design round)**



can you explain below terms in simple language and with example

Ranking requests: ~100–500 QPS peak

Recency

Ranking latency: < 2 seconds, does this mean by any background job to rank ?

Vector search: < 100ms, does this mean to find top 10 by the API ?

Add below Questions which I think may be relevant for any systems

1. Technology Choice or Budget Constraint?
2. Future Scope of the System From Admin or User Side?
3. MultiTenant System, Payment or Subscription Based, MultiLingual or any such Plans?
4. Future Planning, to make the system Live or to test it first with MVP and then decide on the Future Path?
5. GDPR Compliance based on country?

just provide the output for above don't add any bonus item until it's critical for the decision making



Here's a simple, clear explanation of each term + your added questions.

1) Ranking requests: ~100–500 QPS peak

QPS = Queries Per Second

👉 This means:

- At peak time, your system should handle **100 to 500 ranking requests every second**

Example

- A recruiter opens a job → system fetches top 10 candidates → **1 request**
- 200 recruiters doing this at the same time → **200 QPS**

👉 So your system must:

- Respond quickly
 - Not crash under load
-

2) Recency

Recency = how recent or fresh something is

👉 In hiring, newer activity is often more relevant.

Examples

- Candidate updated resume **yesterday** → more relevant
- Candidate updated resume **2 years ago** → less relevant

How it's used in ranking

- Add a boost to recently active candidates

Candidate A:

- Updated 2 days ago → +score

Candidate B:

- Updated 1 year ago → lower score

👉 Helps avoid showing outdated profiles

3) Ranking latency: < 2 seconds

Does this mean background job?

👉 No, not necessarily.

What it means

- When user asks:

"Give me top 10 candidates for this job"

👉 System should respond in **under 2 seconds**

Two ways to achieve this:

Option A: On-demand (real-time)

- Compute ranking when API is called
- Used when:
 - Data changes frequently
 - Need fresh results

✅ Likely expected here

Option B: Precomputed (background job)

- Pre-rank candidates periodically
- Store results
- API just fetches

✅ Faster

❌ Might be slightly outdated

👉 In real systems → **hybrid approach**

- Precompute + slight real-time adjustment
-

4) Vector search: < 100ms

Does this mean finding top 10 via API?

👉 Yes — specifically:

It means:

- The **vector database** should return **top matching candidates quickly (<100ms)**
-

Example flow

1. Convert job description → embedding vector

2. Query vector DB:

"Find top similar candidate vectors"

3. DB returns:

- Top 100 candidates in ~100ms

4. Then:

- Ranking logic picks top 10

👉 Important distinction:

Step	Time
Vector search	<100ms
Full ranking response	<2 seconds

5) Your Additional Questions (Refined for System Design)

These are excellent — here's how to frame them properly:

1. Technology Choice / Budget Constraints

👉 Why it matters:

- Determines:
 - Cloud vs on-prem
 - Managed services vs open-source
 - LLM/API usage cost

Example

- Low budget → FAISS + open-source models
- High budget → Pinecone + OpenAI embeddings

2. Future Scope (Admin & User)

👉 Clarifies system evolution

Admin side

- Analytics dashboard
- Manual ranking overrides
- Bias monitoring

User side (Recruiters)

- Filters (location, salary)
 - Candidate explanations
 - Saved searches
-

3. Multi-tenant / Subscription / Multilingual

👉 Affects core architecture

Multi-tenant

- Multiple companies using same platform
- Requires:
 - Data isolation
 - Tenant-aware ranking

Subscription / Payment

- Feature gating (e.g., premium ranking)

Multilingual

- Resume + JD in different languages
 - Requires:
 - Multilingual embeddings
 - Translation layer
-

4. MVP vs Full Production Plan

👉 Critical for decision-making

MVP (Phase 1)

- Simple pipeline
- Basic ranking (semantic similarity)
- Fast to build

Production (Phase 2+)

- ML improvements
 - Feedback loop
 - Personalization
-

5. GDPR / Compliance

👉 Important for legal + trust

Needs:

- Data deletion (Right to be forgotten)
- Consent for storing resumes
- Data encryption
- Region-based storage

Example

- EU candidate requests deletion → system must remove all traces
-

Summary (Super Simple)

Term	Meaning
QPS	Requests per second system must handle
Recency	Prefer newer/active candidates

Term	Meaning
Ranking latency	Full API response time (<2s)
Vector search	Fast similarity lookup (<100ms)

If you want, next I can:

- Turn this into **interview-ready answers**
- Or map each term to **real architecture decisions**



add the above questions to relevant section under "Open Questions"



Here's your updated **Open Questions** section with your additional points properly integrated and organized.

5) Open Questions

Product Questions

- What defines a "good match"? (skills vs experience vs culture)
- Should recruiters be able to tweak ranking weights?
- Do we need explainability? ("Why this candidate is ranked higher?")
- What features are expected for:
 - Recruiters (filters, saved searches, etc.)
 - Admins (analytics, monitoring, overrides)

Data Questions

- Do we have labeled hiring outcomes (hired / rejected data)?
- Are resumes multilingual?
- Do we need OCR for scanned PDFs?
- How frequently are resumes and job descriptions updated?

ML / Ranking Questions

- Which embedding model should be used (OpenAI, SBERT, custom)?
- Do we fine-tune models or rely on zero-shot initially?
- How do we handle bias in ranking?
- Should we incorporate feedback loops (clicks, hires) into ranking?

System / Architecture Questions

- Should ranking be computed:
 - Fully real-time?
 - Precomputed via background jobs?
 - Hybrid approach?
 - What are the latency expectations for different components?
 - How do we handle scaling as candidates grow from 50K → millions?
-

Technology & Budget Constraints

- What is the budget for infrastructure and AI services?
 - Should we use:
 - Managed services (e.g., hosted vector DBs, APIs)?
 - Open-source/self-hosted solutions?
 - Any constraints on cloud providers or on-prem requirements?
-

Multi-Tenancy / Business Model

- Will the platform support multiple companies (multi-tenant)?
 - How will tenant data be isolated?
 - Is the system:
 - Subscription-based?
 - Pay-per-use?
 - Do we need feature gating for premium users?
-

Internationalization / Localization

- Do we need to support multiple languages for resumes and job descriptions?
- Should we:
 - Translate content?
 - Use multilingual embeddings?

- Are there region-specific requirements?
-

MVP vs Future Roadmap

- Are we building:
 - An MVP first for validation?
 - A full production-ready system from day one?
 - What is the rollout strategy:
 - Internal testing?
 - Limited beta?
 - Full-scale launch?
 - What features are Phase 1 vs Phase 2?
-

Compliance & Legal

- Do we need to comply with regulations like GDPR?
 - Do we need:
 - Data deletion support (Right to be forgotten)?
 - Consent management for storing resumes?
 - Are there country-specific data residency requirements?
 - How do we ensure PII protection and encryption?
-

This version keeps your questions structured and aligned with system design thinking, making it strong for both **interviews and real-world architecture discussions**.



based on the above design a high-level architecture initially for MVP and later on can be extended

Include:

- Components
- Data flow
- Technology choices (".NET + Azure" preferred)

keep it practical for a 4-hour implementation.



Here's a **practical, MVP-first high-level architecture** using **.NET + Azure**, designed so you can realistically build a thin slice in ~4 hours and still have a clear path to scale.

High-Level Architecture (MVP → Extensible)

1) MVP Architecture (4-hour implementable)

Components

1. API Layer

- ASP.NET Core Web API
- Endpoints:
 - Upload Resume
 - Upload Job Description
 - Get Top 10 Candidates

2. Document Storage

- Azure Blob Storage
- Stores raw files (PDF, DOCX)

3. Parser Service (Simple)

- Extract text from documents
- Libraries:

- `UglyToad.PdfPig` (PDF)
- `DocumentFormat.OpenXml` (DOCX)

4. **Embedding Service**

- Calls embedding model (API-based for speed)
- Option:
 - Azure OpenAI embeddings

5. **Vector Store (Simple)**

- In-memory (for MVP) OR
- Azure Cognitive Search (vector support) OR
- Local FAISS (optional if Python interop)

6. **Metadata DB**

- Azure SQL / SQLite (MVP)
- Stores:
 - Candidate info
 - Job info
 - File references

7. **Ranking Service**

- Computes similarity + simple scoring
- Returns Top 10

Data Flow (MVP)

```

[User Upload Resume/JD]
    ↓
[ASP.NET API]
    ↓
[Azure Blob Storage]
    ↓
[Parser Service]
    ↓
[Extracted Text]
    ↓
[Embedding Service (Azure OpenAI)]
  
```

↓
[Vector Stored]
↓
[Metadata Stored in DB]

Ranking Flow

[Get Top Candidates API]
↓
[Fetch Job Embedding]
↓
[Vector Search (Top N candidates)]
↓
[Ranking Service]
↓
[Return Top 10]

Technology Choices (MVP)

Layer	Tech
API	ASP.NET Core
Storage	Azure Blob Storage
DB	Azure SQL / SQLite
Parsing	PdfPig + OpenXML
Embeddings	Azure OpenAI
Vector Search	Azure Cognitive Search (or in-memory)
Hosting	Azure App Service

Why this works for 4 hours

- No microservices → single deployable API
 - Minimal infra setup
 - External AI handles complexity
 - No async pipelines initially
-

2) Extended Architecture (Production-Ready)

Once MVP works, evolve into this:

New Components

1. **API Gateway**
 - Azure API Management
2. **Ingestion Service (Async)**
 - Handles uploads
 - Pushes events
3. **Queue**
 - Azure Service Bus / Queue Storage
4. **Processing Workers**
 - Parse → Extract → Embed
 - Scalable background jobs
5. **Feature Extraction Service**
 - Skills, experience extraction (NER models)
6. **Vector Database**
 - Azure Cognitive Search (vector)
 - Or Pinecone / Weaviate
7. **Ranking Engine (Advanced)**
 - Multi-factor scoring
 - ML-based re-ranking
8. **Cache Layer**
 - Azure Redis
 - Cache Top 10 results

9. Monitoring

- Azure Application Insights

Data Flow (Production)

```
[Upload]
  ↓
[API]
  ↓
[Blob Storage]
  ↓
[Queue (Service Bus)]
  ↓
[Worker Services]
  ↓
[Parsing + NLP + Embeddings]
  ↓
[Vector DB + SQL DB]
```

Query Flow

```
[Get Top Candidates]
  ↓
[Cache Check (Redis)]
  ↓ (miss)
[Vector Search]
  ↓
[Ranking Engine]
  ↓
[Cache Result]
  ↓
[Return Response]
```



Technology Choices (Extended)

Layer	Tech
API Gateway	Azure API Management
Async Messaging	Azure Service Bus
Workers	Azure Functions / Container Apps
Vector DB	Azure Cognitive Search
Cache	Azure Redis
Monitoring	Application Insights
ML Models	Azure OpenAI / Custom

3) Cold Start Handling (Built-in from MVP)

Even in MVP:

- Use **semantic embeddings**
- No training data required
- Ranking based on:
 - Similarity score
 - Basic heuristics

👉 So system works **from Day 1**

4) Minimal Ranking Logic (MVP)

```
Score =  
0.7 * cosine_similarity +  
0.2 * skill_overlap (basic keyword match) +  
0.1 * recency
```

👉 Simple but effective



5) Practical Implementation Plan (4 Hours)

Hour 1

- Setup ASP.NET project
- Create upload + get APIs

Hour 2

- Add parsing (PDF/DOCX)
- Store files in Blob

Hour 3

- Integrate Azure OpenAI embeddings
- Store vectors (in-memory or simple DB)

Hour 4

- Implement cosine similarity
- Return Top 10 candidates

6) Key Tradeoffs (MVP)

Decision	Tradeoff
Sync processing	Simpler but slower ingestion
External embeddings	Fast dev but cost
No queue	Less scalable initially
Simple ranking	Lower accuracy but good baseline

Final Takeaway

- **MVP = simple, synchronous, single service**
- **Production = event-driven, scalable, modular**

- Designed to:
 - Work in 4 hours 
 - Scale to millions later 
-

If you want next, I can:

- Generate **.NET project structure + code**
- Write **actual API endpoints**
- Or create a **ready-to-submit DESIGN.md**



for MVP at initial level thinking of below, validate the pros and cons for it keeping the time constraint in mind

API End Points

Store to BLOB and Publish Message to queue

Function APP handles the message gets the embedding and store it to DB (can we make use of "Cosmos DB" to store both Data and Vector ?



Your approach is actually **very solid for an MVP**, especially given the **4-hour constraint**. It's slightly more advanced than a "basic MVP," but still doable if you keep things minimal.

Let's validate it clearly.

✅ Your Proposed MVP Design

Flow

API → Blob Storage → Queue → Function App → Embedding → DB (Cosmos DB?)

👍 Pros (Why this is a good idea)

1) Asynchronous = Scalable from Day 1

- API is fast → just uploads + pushes message
- Heavy work (parsing + embedding) is offloaded

👉 Prevents API timeout issues

2) Clean Separation of Concerns

- API = ingestion
- Function = processing

👉 Easier to extend later into microservices

3) Queue Adds Reliability

- If Function fails → message retries
- No data loss

👉 This is a **big upgrade over sync MVP**

4) Closer to Production Design

- You're not throwing away MVP later
 - This design scales naturally
-

5) Cosmos DB for Unified Storage (Good Thought 👍)

👉 Yes, you **can use Cosmos DB for both metadata + vectors**

- Cosmos DB (with vector indexing support) allows:
 - Store JSON (candidate profile)
 - Store embedding vector
 - Run similarity queries
-

⚠️ Cons (Important for 4-hour constraint)

1) Setup Overhead (Biggest Risk)

You'll need to configure:

- Azure Blob
- Azure Queue / Service Bus
- Azure Function
- Cosmos DB

👉 This alone can consume **1.5–2 hours**

2) Debugging Complexity

If something breaks:

- Is it API?
- Queue?
- Function?
- Cosmos DB?

👉 Harder to debug under time pressure

3) Cosmos DB Vector Search Limitations (Important)

- Vector search in Cosmos DB:
 - Not as mature as dedicated vector DBs
 - Requires proper indexing config

👉 Might slow you down if unfamiliar

4) Cold Start (Function App)

- Azure Functions may have **cold start delay**
 - Not ideal for demo responsiveness
-

5) More Moving Parts = More Failure Points

Compared to simple MVP:

- More infra → more chances of misconfiguration
-

🤔 Should You Use Cosmos DB for Vectors?

✅ **YES — if you want simplicity**

Pros

- Single database (no extra infra)
- JSON + vector in same document
- Easy to query

Cons

- Less optimized than vector DB
- Slightly more setup

💡 Recommendation (Very Practical)

If time is tight (safer path)

Use Cosmos DB BUT:

- Skip advanced vector indexing initially
- Store embeddings as array
- Do similarity in application code (small dataset = fine)

👉 This avoids config complexity

⚖️ Final Verdict (Brutally Honest)

Your Design Rating for MVP:

Aspect	Verdict
Architecture quality	★★★★★ (Very good)
Scalability	★★★★★
4-hour feasibility	⚠️ Medium risk
Simplicity	❌ Moderate complexity

🟢 Suggested “Balanced MVP” (Best of both)

If you want to **stay safe within 4 hours**, slightly simplify:

Option A (Recommended)

API → Blob → Queue → Function → Cosmos DB

BUT:

- Keep Function logic very simple
- Avoid advanced indexing
- Limit features

Option B (Ultra-safe fallback)

API → Blob → **Sync** processing → Cosmos DB

👉 Use this if:

- You run out of time
- Queue setup becomes slow

🚀 Key Advice (Critical)

👉 Prioritize **working demo over perfect architecture**

In 4 hours, interviewer values:

- End-to-end working flow ✅
- Clean reasoning ✅
- Tradeoff awareness ✅

More than:

- Fully production-grade infra ❌



Final Answer (Short Summary)

- Your design is **strong and scalable**
- Cosmos DB **can be used for both data + vectors**
- Biggest risk = **setup complexity + time**
- Keep implementation **minimal and focused**

If you want next, I can:

- Give you **exact Azure resources + minimal config**
- Or **sample .NET code for queue + function + Cosmos DB**



more information on cosmosDB indexing part and simple example explanation of how it will work and will calculate the ranking for a JD with thousands of resume Vectors stored in cosmosDB?
any loophole in this MVP which should be avoided



Great question — this is exactly where most MVP designs either **succeed or silently fail**. I'll explain this in **very practical terms** without over-complicating it.

1) Cosmos DB Indexing (Simple Explanation)

What is indexing in Cosmos DB?

👉 Indexing = how fast Cosmos DB can **find data**

By default:

- Cosmos DB indexes **JSON fields** (like name, skills, etc.)

What about vectors?

Cosmos DB now supports **vector indexing**, which allows you to:

- Store embeddings (arrays like `[0.12, 0.98, ...]`)
- Run **similarity search (cosine distance)** directly in DB

Two Ways You Can Use It (MVP vs Advanced)

✅ Option 1: SIMPLE (Recommended for MVP)

👉 Store embedding like this:

```
{  
  "id": "candidate-123",
```

```
"name": "John Doe",
"skills": ["C#", "Azure"],
"embedding": [0.12, 0.98, 0.33, ...],
"updatedAt": "2026-04-05"
}
```

Indexing

- Keep **default indexing**
- DO NOT configure vector index initially

How search works

1. Get JD embedding
2. Fetch candidates (e.g., 1000)
3. Compute similarity in your .NET code

Example (C# pseudo)

```
foreach (var candidate in candidates)
{
    var score = CosineSimilarity(jobEmbedding, candidate.embedding);
}
```

👉 Then sort → Top 10

Why this works

- 50K candidates = manageable
- Simple to implement
- No Cosmos DB complexity

⚡ Option 2: VECTOR INDEXING (Advanced)

👉 Cosmos DB allows queries like:

```
SELECT TOP 10 *  
FROM c  
ORDER BY VectorDistance(c.embedding, @jobEmbedding)
```

Required setup

You must define vector index policy like:

```
{  
  "vectorIndexes": [  
    {  
      "path": "/embedding",  
      "type": "cosine"  
    }  
  ]  
}
```

Pros

- Fast (no manual loop)
- Scales better

Cons (Important for MVP)

- Setup complexity
- Easy to misconfigure
- Hard to debug

🔍 2) How Ranking Works with Thousands of Resumes

Let's walk through a **real example**.

Scenario

- 10,000 resumes stored
 - Each has embedding vector
 - New Job Description arrives
-

Step-by-step flow

Step 1: Convert JD → embedding

JD: "Looking for .NET developer with Azure experience"

↓

[0.21, 0.77, 0.11, ...]

Step 2: Compare with all candidates

For each candidate:

Candidate A → [0.20, 0.75, 0.10]

Candidate B → [0.90, 0.10, 0.40]

Step 3: Compute similarity

JD vs A → 0.95 ✓ (very similar)

JD vs B → 0.30 ✗ (not relevant)

Step 4: Rank

Top candidates:

1. Candidate **A** (0.95)
2. Candidate **C** (0.91)
3. Candidate **D** (0.88)
- ...

Step 5: Return Top 10

⚠ Important Optimization (MVP)

👉 DO NOT scan all 50K every time

Instead:

- Filter first:
 - Skills contains ".NET"
- Then compute similarity

🔴 3) Hidden Loopholes in Your MVP (Very Important)

These are the **real pitfalls** you should avoid.

❌ 1) Full Scan Problem (Biggest Risk)

Problem

- Fetching all candidates → slow

Fix

- Pre-filter:
 - Skills
 - Experience range

👉 Reduces from 50K → ~1K

❌ 2) Large Payload from Cosmos DB

Problem

- Embeddings are large arrays
- Fetching many = heavy network cost

Fix

- Use projection:

```
SELECT c.id, c.embedding FROM c
```

❌ 3) RU (Request Units) Explosion

Problem

- Cosmos DB charges per query
- Full scans = expensive

Fix

- Use partitioning:
 - e.g., partition by `skill` or `tenantId`

❌ 4) No Retry / Poison Message Handling (Queue)

Problem

- Failed Function → lost processing

Fix

- Enable:
 - Retry policy
 - Dead-letter queue

❌ 5) Embedding Version Drift

Problem

- If you change model later → old vs new embeddings mismatch

Fix

Store:

```
"embeddingVersion": "v1"
```

❌ 6) Cold Start Delay (Function App)

Problem

- First request slow

Fix

- Use:
 - Premium plan OR
 - Keep-alive ping

❌ 7) No Idempotency

Problem

- Same resume processed twice

Fix

- Use unique ID + upsert

❌ 8) Ranking Quality Issues

Problem

- Pure similarity is not enough

Fix (simple MVP)

Add:

```
Score =  
0.8 * similarity +  
0.2 * skill match
```

✅ Final Recommendation (Very Practical)

For YOUR MVP:

👉 Use this approach:

- Cosmos DB:
 - Store embeddings
 - No vector indexing initially
- Query:
 - Filter candidates
 - Compute similarity in .NET
- Queue + Function:
 - Keep simple

When to Upgrade Later

Move to:

- Cosmos vector indexing OR
- Azure Cognitive Search

👉 Only when:

- Candidates > 100K
- Latency becomes issue



Final Takeaway

- Cosmos DB **can store vectors easily**
 - For MVP → **avoid advanced vector indexing**
 - Ranking = **compute similarity in app layer**
 - Biggest risk = **full scan + RU cost**
-

If you want next, I can:

- Give you **exact Cosmos DB schema + partition strategy**
- Or write **C# cosine similarity + query code**



here pdf will be scanned and the whole text can be stored now how Skills can be fetched from the whole text of pdf, will it add more complexity ?



Short answer: **Yes, it adds some complexity — but you can keep it very manageable for MVP** by choosing the right approach.

Let's break it down simply and practically.

1) What's the Problem?

You now have:

- Full resume text (after parsing PDF)

👉 Example:

```
"John Doe is a software engineer with 5 years of experience in C#, .NET, Azure, SQL Server, and React..."
```

But your system needs:

```
{  
  "skills": ["C#", ".NET", "Azure", "SQL Server", "React"]  
}
```

2) Ways to Extract Skills (From Simple → Advanced)

Option 1: Keyword Matching (BEST for MVP)

👉 Maintain a predefined **skills dictionary**

Example

```
var skillsList = new List<string>
{
    "C#", ".NET", "Azure", "SQL", "React", "Java", "Python"
};
```

Logic

```
var extractedSkills = skillsList
    .Where(skill => resumeText.Contains(skill,
StringComparison.OrdinalIgnoreCase))
    .ToList();
```

Output

```
["C#", ".NET", "Azure", "React"]
```

✓ Pros

- Very fast to implement (15–20 mins)
- No ML required
- Works reasonably well

✗ Cons

- Misses variations:
 - "Dot Net" vs ".NET"
 - "MS Azure" vs "Azure"

👉 **Verdict: PERFECT for 4-hour MVP**

⚡ **Option 2: Regex + Heuristics**

👉 Slight improvement over keyword matching

Example

- Normalize text:
 - "dot net" → ".net"
 - "node js" → "node.js"
-

Pros

- Better accuracy than plain matching

Cons

- More effort
 - Still rule-based
-

🤖 **Option 3: NLP / AI Extraction (Advanced)**

Use:

- Azure OpenAI
- Named Entity Recognition (NER)

Prompt example

Extract all technical skills from this resume:
<resume text>

Output

```
["C#", ".NET", "Azure"]
```

Pros

- Very accurate
- Handles variations

Cons

- Slower
- Costs money
- Overkill for MVP



3) Recommended Approach for YOUR MVP

👉 Use a **hybrid simple approach**

Step 1: Normalize text

```
text = text.ToLower();
```

Step 2: Keyword match

Use a **small curated list (50–100 skills)**

Step 3: Store skills in Cosmos DB

```
{  
  "id": "candidate-123",
```



```
"skills": ["c#", ".net", "azure"],  
"embedding": [...]  
}
```

4) Why Skills Matter (Important for Your Design)

Skills help you:

1. Pre-filter candidates

Instead of:

Compare 50,000 candidates ❌

Do:

Filter "Azure" → 2,000 candidates ✅

2. Improve ranking

```
Score =  
0.7 * similarity +  
0.3 * skill_overlap
```

3. Reduce Cosmos DB cost (RU)

👉 Smaller dataset → cheaper queries

5) Complexity Impact (Honest Answer)

Approach	Complexity	Time
Keyword matching	🟢 Low	20–30 min
Regex improvements	🟡 Medium	+30 min
AI extraction	🔴 High	Not MVP-friendly

👉 So:

Yes, it adds complexity — but only if you over-engineer it.

⚠️ 6) Common Mistakes to Avoid

❌ 1) Trying to be “too smart” early

- Don’t build ML pipeline
- Don’t fine-tune models

❌ 2) Huge skill dictionary

- Keep it small (top common skills)

❌ 3) Ignoring normalization

Bad:

"Azure" ≠ "azure"

❌ 4) Not storing extracted skills

👉 Always persist → reuse for ranking

✅ 7) Final Recommendation

For your MVP:

👉 Do this:

1. Parse PDF → text
2. Normalize text
3. Extract skills using keyword list
4. Store in Cosmos DB
5. Use skills for:
 - Filtering
 - Ranking boost



Final Takeaway

- Skill extraction **does add a step**, but:
 - It's **simple to implement**
 - It **massively improves performance + quality**

👉 Think of it as:

"Cheap intelligence layer" before AI kicks in

If you want next, I can:

- Give you a **ready-to-use skill dictionary**
- Or write **clean C# utility for extraction + normalization**





It violates the actual problem statement of keyword searching instead of semantic search
is CosmosDB overkill here ?
and if going with Cosmos will indexing help and how ?



You're asking the **right architectural question** — this is exactly where many solutions drift away from the core goal.

Let's address each concern clearly and practically.



1) "Does skill extraction violate semantic search?"

👉 **No — if used correctly. Yes — if misused.**

❌ Wrong approach (violates requirement)

If you do:

Match candidates **ONLY** based on keyword/skills

👉 That's basically your **old system again**



✅ Correct approach (what you should do)

Use **semantic search as primary**, and **skills as a supporting signal**

Final Score =

0.8 * semantic_similarity ✅ main driver

0.2 * skill_overlap ✅ helper



Why this is valid

- Embeddings capture meaning:
 - "Backend engineer" $\approx$ "API developer"
- Skills help:
 - Filter noise
 - Improve precision

👉 Think of skills as:

"Optimization + signal boost", NOT the core logic



2) Is Cosmos DB Overkill for MVP?

👉 **Short answer: Slightly yes — but acceptable depending on your goal**

When it *feels like overkill*

For your scale:

- 50K candidates
- Single use-case
- 4-hour MVP

👉 You could easily use:

- In-memory list OR
 - Simple SQL
-

When it *makes sense*

Use Cosmos DB if you want:

- Cloud-native demo
 - Future scalability
 - JSON flexibility
 - Multi-tenant readiness
-



Practical Verdict

Goal	Recommendation
Just pass assignment fast	✗ Cosmos not needed
Show good system design thinking	✓ Cosmos is good
Balance both	⚖ Use Cosmos simply (no advanced features)

3) Will Cosmos DB Indexing Help?

👉 Yes — but NOT in the way you might expect for vectors

Let's break it down.

A. What Cosmos Indexing Helps With

Cosmos DB indexing is great for:

1. Filtering

```
SELECT * FROM c WHERE ARRAY_CONTAINS(c.skills, "azure")
```

👉 Fast because of indexing

2. Metadata Queries

```
WHERE c.experience > 3
```

👉 This is VERY useful for your system because:

Optimization Strategy

Step 1: Filter candidates (skills, experience) → 1000

Step 2: Apply semantic similarity → Top 10

👉 This is where Cosmos indexing shines

B. What Cosmos Indexing DOES NOT (fully) solve in MVP

✗ It does NOT automatically give you:

- Efficient large-scale vector similarity (unless you configure vector indexing)
- ANN (Approximate Nearest Neighbor) like FAISS/Pinecone

C. Cosmos Vector Indexing (Reality Check)

Yes, Cosmos supports:

```
ORDER BY VectorDistance(c.embedding, @vector)
```

BUT:

⚠ Limitations for MVP

- Setup complexity
- Less mature than dedicated vector DBs
- Harder to debug quickly

👉 So:

! For MVP:

Indexing helps filtering, NOT core ranking

🧩 4) How It All Fits Together (Correct Design)

✅ Recommended Flow

1. JD → embedding
2. Cosmos DB query:
Filter candidates
(skills / experience / location)
3. Application layer:
Compute cosine similarity
4. Rank + return Top 10

Why this is ideal

Component	Role
Cosmos DB	Fast filtering
Embeddings	Semantic understanding
App layer	Final ranking

🚨 5) Biggest Pitfall to Avoid

👉 This is critical:

❌ Don't do this:

Cosmos DB → **fetch ALL** candidates → rank

👉 Problems:

- Slow
- Expensive (RU cost)
- Not scalable

✅ Do this instead:

Cosmos DB → filtered **subset** → rank

🎯 6) Final Recommendation (Clear Answer)

On Skills vs Semantic

- ✅ Skills = supporting signal
- ❌ Not the primary matching logic

On Cosmos DB

- Slight overkill for MVP
- But:
 - ✅ Good for demonstrating architecture
 - ✅ Useful for filtering + storage

On Indexing

- ✅ Helps filtering performance significantly
- ❌ Does NOT replace vector search (in MVP setup)



Final Takeaway

- You are still building a **semantic search system**
- Skills are just a **performance + precision layer**
- Cosmos DB:
 - Good for **structured filtering**
 - Not your **core ranking engine**

If you want next, I can:

- Show **exact Cosmos query + C# code for filtered retrieval**
- Or help you **decide between Cosmos vs Azure Cognitive Search (vector-first design)**



Great comparison — this is a **key architectural decision** and directly impacts how “AI-native” your system really is.

I’ll keep it **practical + MVP-focused**, not theoretical.

1) Core Difference (Simple)

Aspect	Cosmos DB	Azure Cognitive Search
Primary purpose	General-purpose NoSQL DB	Search + ranking engine
Strength	Storing data	Finding best matches
Vector capability	Added feature	Core feature

👉 Think:

- **Cosmos DB** = “Store everything”
- **Azure Cognitive Search** = “Find best results fast”

2) How Each Fits Your Use Case

✅ Cosmos DB Approach (What you’re doing)

Flow

Store → **Filter** → App computes similarity → Rank

Responsibilities

Layer	Responsibility
Cosmos DB	Store resumes + embeddings
App (.NET)	Compute similarity + ranking

Azure Cognitive Search Approach (Vector-first) Flow

Store → Search engine computes similarity → Returns Top K

Responsibilities

Layer	Responsibility
Cognitive Search	Vector search + ranking
App	Minor post-processing

3) Example Comparison

Cosmos DB

Step 1: Fetch candidates

```
SELECT c.id, c.embedding
FROM c
WHERE ARRAY_CONTAINS(c.skills, "azure")
```

Step 2: In your .NET app

```
var score = CosineSimilarity(jobEmbedding, candidate.embedding);
```

Step 3: Sort + Top 10



Azure Cognitive Search

Single query does everything:

```
{
  "vector": {
    "value": [0.21, 0.77, ...],
    "k": 10,
    "fields": "embedding"
  }
}
```

👉 Returns **Top 10 directly**

⚡ 4) Performance Comparison

Factor	Cosmos DB	Cognitive Search
Vector search speed	❌ Slower (manual)	✅ Fast (optimized ANN)
Scaling	⚠️ App burden	✅ Built-in
Latency	Higher	Lower
Dev effort	Medium	Slightly higher setup



5) MVP Tradeoff (Very Important)



Cosmos DB (Your Current Path)



✓ Pros

- Simple mental model
- One DB for everything
- Easy to start
- Flexible schema

✗ Cons

- You implement ranking logic
 - Slower as data grows
 - RU cost if not filtered well
-

■ Azure Cognitive Search

✓ Pros

- True semantic/vector search
- Built-in ranking
- Fast (sub-100ms)
- No manual similarity code

✗ Cons

- Setup takes time
 - Slight learning curve
 - Another service to manage
-

🎯 6) Which Should YOU Choose (4-hour reality)

🟢 If your goal is:

✓ "Finish MVP reliably in 4 hours"

👉 Go with **Cosmos DB**

-
- 🟢 If your goal is:
 - ✅ "Show strong AI-native system design"

👉 Go with **Azure Cognitive Search**

⚖️ 7) Best Practical Hybrid (Highly Recommended)

👉 This is what strong candidates do:

💡 Hybrid Design

```
Blob → Queue → Function
      ↓
Store in Cosmos DB (metadata)
      ↓
Store in Cognitive Search (embedding index)
```

Why this is powerful

Component	Role
Cosmos DB	Source of truth
Cognitive Search	Fast retrieval

Query flow

1. JD → embedding
2. Cognitive Search → Top 50 candidates
3. Cosmos DB → fetch details
4. App → final ranking

👉 Best of both worlds

8) Common Mistake to Avoid

❌ Using Cosmos DB like a search engine

- Full scans
- Manual ranking at scale

👉 This will break at ~100K+ data

❌ Overengineering MVP

- Don't integrate both systems unless confident with time
-



9) Final Recommendation (Clear & Practical)

For YOUR case:

🟢 Safe MVP

👉 Cosmos DB + App-based similarity

🟢 Stronger MVP (if time permits)

👉 Azure Cognitive Search (vector search)

🟢 Ideal Production

👉 Hybrid (Cosmos + Cognitive Search)

Final Takeaway

- **Cosmos DB = storage + filtering**
 - **Cognitive Search = ranking engine**
 - If you want true **AI-native system** → **vector search should not live in your app**
-

If you want next, I can:

- Give you **exact Azure Cognitive Search index schema**
- Or **.NET code to query vector search API**



let's choose only cosmosDB to keep and later on add Cognitive Search for performance and cost optimization

so final will be

Store → Filter → App computes similarity → Rank

Design the end-to-end pipeline

- File ingestion (PDF/DOCX)
- Parsing
- Embedding generation
- Storage
- Matching

Keep it scalable but simple



Great — this is a **clean, practical decision**.

You're keeping MVP simple with **Cosmos DB + app-based semantic ranking**, while leaving room to plug in Cognitive Search later.

Below is a **clear, end-to-end pipeline design** aligned with your choice:

Final MVP Architecture (Cosmos-first)

Upload → Blob → Queue → Function → Cosmos DB



Matching API (.NET)



Top 10 Candidates

1) Components

1. API Layer (.NET - ASP.NET Core)

- Upload Resume / JD
- Get Top 10 Candidates

2. Azure Blob Storage

- Stores raw files (PDF/DOCX)
 - Acts as source of truth for documents
-

3. Queue (Azure Queue / Service Bus)

- Decouples ingestion from processing
 - Ensures reliability
-

4. Azure Function (Processing Worker)

Handles:

- Parsing
 - Skill extraction (simple)
 - Embedding generation
-

5. Cosmos DB

Stores:

- Candidate metadata
 - Extracted skills
 - Embeddings
-

6. Matching Service (.NET API)

- Fetch candidates (filtered)
 - Compute similarity
 - Rank Top 10
-

2) End-to-End Data Flow

Step 1: File Ingestion Flow

```
User uploads Resume / JD
      ↓
ASP.NET API
      ↓
Store in Azure Blob
      ↓
Publish message to Queue
```

Queue Message Example

```
{
  "documentId": "123",
  "type": "resume",
  "blobUrl": "https://blob/.../resume.pdf"
}
```

Step 2: Parsing (Function App)

Function triggers on queue message

PDF/DOCX Parsing

Use:

- PDF → PdfPig
- DOCX → OpenXML

Output

Raw extracted text

Important

- Keep parsing simple
- Ignore formatting issues in MVP

Step 3: Skill Extraction (Lightweight)

Extract skills **using** keyword list

Example:

Input:

"C# developer with Azure and SQL"

Output:

["c#", "azure", "sql"]

Step 4: Embedding Generation

Use:

- Azure OpenAI Embeddings

Flow

Resume Text → Embedding API → Vector

Example:

```
[0.21, 0.77, 0.11, ...]
```

Store embedding version

```
"embeddingVersion": "v1"
```

Step 5: Storage (Cosmos DB)

Document Structure

```
{
  "id": "candidate-123",
  "type": "candidate",
  "name": "John Doe",
  "skills": ["c#", "azure", "sql"],
  "experienceYears": 5,
  "embedding": [0.21, 0.77, ...],
  "updatedAt": "2026-04-05",
  "embeddingVersion": "v1"
}
```

Partition Key (IMPORTANT)

👉 Use:

```
/partitionKey = "candidate"
```


OR better:

```
/partitionKey = skill or tenantId (future-ready)
```

3) Matching Flow (Core Logic)

Step 1: JD Processing

- Parse JD
- Extract skills
- Generate embedding

Step 2: Filter Candidates (Cosmos DB)

```
SELECT c.id, c.embedding, c.skills  
FROM c  
WHERE ARRAY_CONTAINS(c.skills, "azure")
```

👉 Reduces dataset from 50K → ~1K

Step 3: Compute Similarity (.NET)

```
double score = CosineSimilarity(jobEmbedding, candidateEmbedding);
```

Step 4: Final Score

```
Score =  
0.8 * semantic_similarity +
```

```
0.2 * skill_overlap
```

Step 5: Rank

- Sort by score
- Return Top 10

4) Scaling Considerations (Still Simple)

A. Avoid Full Scan

 Don't:

```
Fetch all 50K candidates
```

 Do:

```
Filter → then rank
```

B. Limit Candidates for Ranking

```
Fetch max 500-1000 candidates
```

C. Parallel Similarity Computation

```
Parallel.ForEach(candidates, candidate =>
{
```

```
    ComputeSimilarity(...)  
});
```

D. Cache (Optional but easy)

- Cache Top 10 results for a JD (Redis or in-memory)



5) Failure Handling (Keep it Minimal)

Problem	Solution
Function fails	Queue retry
Duplicate processing	Upsert in Cosmos
Bad parsing	Store raw text anyway
Embedding API fails	Retry (simple retry logic)



6) Future Extension (Easy Upgrade Path)

Add Azure Cognitive Search later:

Cosmos DB → **source** of truth
Cognitive Search → vector index

New flow:

JD → Cognitive Search → **Top 50**
→ Cosmos DB → **details**
→ App → final ranking





Final Summary

Your Pipeline

```
Upload → Blob → Queue → Function
      ↓
    Parse + Skills + Embedding
      ↓
    Cosmos DB
      ↓
  Filter → Similarity → Rank → Top 10
```

Why this works

- ✓ Simple enough for 4 hours
- ✓ Scalable (async + queue)
- ✓ Semantic-first (not keyword-based)
- ✓ Easy to upgrade later

If you want next, I can:

- Write **actual .NET API + Function code skeleton**
- Or provide **Cosmos DB queries + C# similarity implementation**





now the infra part and implementation is clear
summarize based on the above discussions and requirements

- Design decisions
- Trade-offs
- Limitations
- Improvements if given more time



Here's a **clean, interview-ready summary** of your design based on everything we discussed.

1) Design Decisions

Architecture Style

- **Event-driven pipeline**
 - API → Blob → Queue → Function → Cosmos DB
 - Decouples ingestion from processing → improves reliability & scalability
-

Storage Choice: Cosmos DB

- Used as **single source of truth**
 - Stores:
 - Candidate metadata
 - Extracted skills
 - Embeddings (vectors)
 - Chosen for:
 - Schema flexibility (JSON)
 - Easy scaling
 - Cloud-native integration
-

✅ Semantic-First Matching

- Core ranking based on:
 - **Embedding similarity (primary signal)**
 - Skills used only for:
 - Filtering
 - Minor ranking boost
-

✅ Hybrid Ranking Approach (App-driven)

`Filter` (Cosmos DB) → Similarity (App) → Rank

- Avoids full dataset scan
 - Keeps MVP simple without vector DB complexity
-

✅ Asynchronous Processing

- Queue + Function used for:
 - Parsing
 - Skill extraction
 - Embedding generation
-

✅ Lightweight Skill Extraction

- Keyword-based extraction
 - Used as:
 - Pre-filter
 - Secondary ranking signal
-

✅ Extensibility

- Designed to later plug in:
 - Azure Cognitive Search (vector engine)
 - ML-based ranking improvements



2) Trade-offs



Simplicity vs Performance

Choice	Trade-off
App-based similarity	Simple but slower at scale
No vector DB	Faster dev, less optimized search



Cosmos DB vs Specialized Search

Choice	Trade-off
Cosmos DB only	Fewer components, but weaker vector search
No Cognitive Search	Simpler MVP, less efficient ranking



Skill Extraction Approach

Choice	Trade-off
Keyword matching	Fast, but less accurate
No NLP/LLM	Saves time, reduces quality slightly



Async Pipeline

Choice	Trade-off
Queue + Function	Scalable, but more setup complexity



Filtering Before Ranking

Choice	Trade-off
Pre-filter using skills	Faster queries, but risk of missing candidates

3) Limitations (Current MVP)

1) No True Vector Indexing

- Similarity computed in application layer
 - Not optimized for large-scale datasets (>100K)
-

2) Limited Matching Accuracy

- No:
 - Context-aware ranking
 - Career trajectory understanding
 - Deep NLP extraction
-

3) Skill Extraction is Basic

- Keyword-based:
 - Misses synonyms
 - Sensitive to wording variations
-

4) Potential Bias in Filtering

- Pre-filtering may:
 - Exclude relevant candidates unintentionally
-

❌ 5) Latency May Increase with Scale

- As candidates grow:
 - More data fetched
 - More similarity computations
-

❌ 6) No Feedback Loop

- System does not learn from:
 - Recruiter clicks
 - Hiring decisions
-

❌ 7) Cosmos DB Cost (RU)

- Poor query design can lead to:
 - High RU consumption
 - Expensive scaling
-

🚀 4) Improvements (If Given More Time)

◆ 1) Introduce Vector Search Engine

- Integrate **Azure Cognitive Search**
 - Offload:
 - Similarity search
 - Top-K retrieval
-

◆ 2) Replace Skill Extraction with NLP

- Use:
 - LLM-based extraction OR
 - Pretrained NER models

◆ 3) Advanced Ranking Model

- Move from heuristic:

0.8 similarity + 0.2 skills

👉 To:

- ML-based ranking (learning-to-rank)
- Feature-rich scoring

◆ 4) Feedback Loop

- Capture:
 - Clicks
 - Shortlists
 - Hires

👉 Use for:

- Model improvement
- Personalization

◆ 5) Caching Layer

- Add Redis:
 - Cache Top 10 results
 - Reduce repeated computation

◆ 6) Better Filtering Strategy

- Replace strict skill filtering with:
 - Semantic pre-filter (ANN search)
 - Soft filtering

◆ 7) Multi-Tenant Support

- Partition data by:
 - TenantId
 - Add access isolation
-

◆ 8) Observability

- Add:
 - Metrics (latency, accuracy)
 - Logging
 - Tracing (Application Insights)
-

◆ 9) Compliance & Security

- Add:
 - Data encryption
 - PII masking
 - GDPR compliance features
-



Final One-Line Summary



Built a scalable, semantic-first talent ranking system using Cosmos DB and async processing, prioritizing simplicity for MVP while enabling future evolution into a vector-native architecture.

If you want, I can convert this into a **ready-to-submit DESIGN.md** or **interview pitch (2–3 min explanation)**.





yes design.md and include below and add if anything is missing

- Architecture
- Data flow
- Scaling approach
- Failure modes



Here's a **clean, submission-ready** `DESIGN.md` you can directly use.



DESIGN.md

Talent Intelligence & Ranking Engine (MVP)

1. Overview

This system replaces a legacy keyword-based recruitment engine with a **semantic, AI-driven matching system** that ranks candidates against job descriptions.

The MVP focuses on:

- End-to-end pipeline (ingestion → parsing → embedding → ranking)
- Semantic similarity using embeddings
- Scalable architecture with minimal complexity
- Cloud-native implementation using **.NET + Azure**

2. Architecture

High-Level Architecture

[Client / Recruiter]



[ASP.NET API]



```
[Azure Blob Storage] ← (Raw Files)
    ↓
[Queue (Azure Service Bus / Queue)]
    ↓
[Azure Function (Worker)]
    ↓
[Cosmos DB]
    ↓
[Matching API (.NET)]
    ↓
[Top 10 Candidates Response]
```

Components

1. API Layer (ASP.NET Core)

- Upload resumes and job descriptions
- Fetch Top 10 candidates
- Stateless, horizontally scalable

2. Azure Blob Storage

- Stores raw documents (PDF, DOCX)
- Acts as durable file storage

3. Queue (Azure Queue / Service Bus)

- Decouples ingestion from processing
- Enables retry and reliability

4. Azure Function (Processing Worker)

Handles:

- Document parsing

- Skill extraction (lightweight)
 - Embedding generation (via Azure OpenAI)
-

5. Cosmos DB

Stores:

- Candidate profiles
 - Extracted skills
 - Embeddings (vectors)
 - Metadata (timestamps, versioning)
-

6. Matching Service (.NET API)

- Fetches filtered candidates
 - Computes similarity (cosine similarity)
 - Ranks and returns Top 10
-

3. Data Flow

A. Ingestion Pipeline

1. User uploads Resume / JD
 2. API stores file in Blob Storage
 3. API publishes message to Queue
 4. Function processes message:
 - Parse document → extract text
 - Extract skills (keyword-based)
 - Generate embedding
 5. Store structured data in Cosmos DB
-

B. Matching Pipeline

1. Job Description received
2. Parse + generate embedding
3. Query Cosmos DB (filter candidates)
4. Compute similarity in application layer
5. Rank candidates
6. Return Top 10

C. Data Model (Cosmos DB)

```
{
  "id": "candidate-123",
  "type": "candidate",
  "name": "John Doe",
  "skills": ["c#", "azure", "sql"],
  "experienceYears": 5,
  "embedding": [0.21, 0.77, ...],
  "updatedAt": "2026-04-05",
  "embeddingVersion": "v1"
}
```

4. Ranking Logic

Scoring Function

```
Score =
  0.8 * semantic_similarity +
  0.2 * skill_overlap
```

Matching Steps

1. Generate embedding for JD
 2. Filter candidates (skills / metadata)
 3. Compute cosine similarity
 4. Rank and return Top 10
-

5. Scaling Approach

A. Horizontal Scaling

- API and Function Apps are stateless → scale out easily
 - Azure handles auto-scaling
-

B. Asynchronous Processing

- Queue decouples ingestion from processing
 - Prevents API bottlenecks
-

C. Efficient Querying

- Use Cosmos DB filtering:
 - Skills
 - Experience
 - Reduce dataset before similarity computation
-

D. Controlled Candidate Set

- Limit candidates for ranking:
 - Fetch top 500–1000 candidates only
-

E. Parallel Processing

- Use parallel computation for similarity scoring

F. Future Scaling Path

- Introduce vector search engine (Azure Cognitive Search)
 - Move similarity computation out of application layer
-

6. Failure Modes & Mitigations

Failure Mode	Impact	Mitigation
Parsing failure	Missing data	Store raw text, retry parsing
Queue message loss	Data inconsistency	Use retry + dead-letter queue
Function failure	Processing delay	Automatic retries
Embedding API failure	Missing embeddings	Retry with exponential backoff
Duplicate processing	Data duplication	Use idempotent upsert in Cosmos DB
High latency (large dataset)	Slow ranking	Pre-filter candidates
High RU consumption	Increased cost	Optimize queries + projections
Cold start (Function App)	Delayed processing	Use warm instances / premium plan

7. Design Decisions

- **Semantic-first ranking** using embeddings
 - **Cosmos DB as unified storage**
 - **Queue-based async pipeline** for ingestion
 - **Application-layer ranking** to simplify MVP
 - **Skill extraction as supporting signal (not primary)**
-

8. Trade-offs

Decision	Trade-off
No vector DB	Simpler MVP, but less efficient at scale

Decision	Trade-off
App-based similarity	More control, but higher compute cost
Keyword skill extraction	Fast, but less accurate
Cosmos DB only	Fewer components, but weaker search capabilities

9. Limitations

- No native vector indexing (manual similarity computation)
- Limited NLP capabilities (basic skill extraction)
- Potential filtering bias (missing relevant candidates)
- No learning/feedback loop
- Latency increases as dataset grows
- Cosmos DB RU cost can rise with inefficient queries

10. Future Improvements

- Integrate **Azure Cognitive Search** for vector search
- Replace keyword extraction with NLP/LLM-based parsing
- Introduce ML-based ranking (learning-to-rank)
- Add feedback loop (clicks, hires)
- Add caching layer (Redis)
- Improve filtering with semantic pre-selection
- Support multi-tenancy and access isolation
- Add observability (metrics, logs, tracing)
- Implement compliance features (data privacy, deletion)

11. Conclusion

This MVP delivers a **semantic, scalable candidate ranking system** with:

- Clean separation of ingestion and processing
- Efficient filtering + ranking pipeline

- Minimal infrastructure complexity
- Clear upgrade path to a vector-native architecture

It balances **speed of implementation (4-hour constraint)** with **production-aligned design principles**.



for implementing the solution architecture is clean architecture a best fit ?



Short answer: **Yes — but only a *lightweight* version of Clean Architecture is the best fit for your MVP.**

Full-blown Clean Architecture would be **overkill for a 4-hour implementation.**

Let's break this down clearly.

1) Is Clean Architecture a Good Fit Here?

👉 **Yes, conceptually it fits very well**, because your system has:

- Clear separation of concerns:
 - Ingestion
 - Processing
 - Storage
 - Matching
- External dependencies:
 - Azure Blob
 - Cosmos DB
 - OpenAI embeddings
- Business logic:
 - Ranking
 - Skill extraction

👉 Clean Architecture helps you:

- Keep business logic independent
- Swap infra later (e.g., Cosmos → Cognitive Search)

2) But... Full Clean Architecture?

If you try to implement:

- Domain layer
- Application layer
- Infrastructure layer
- Interfaces
- Dependency inversion everywhere

👉 You will:

- Spend too much time on structure
- Not finish the MVP

✅ 3) Recommended Approach: "Pragmatic Clean Architecture"

👉 Use **simplified Clean Architecture (thin layering)**

Suggested Structure (Perfect for your case)

```
/src
├─ API (Controllers)
├─ Application (Core logic)
├─ Infrastructure (Azure integrations)
└─ Domain (Models)
```

🧩 4) Layer Responsibilities (Simple Version)

■ 1. Domain Layer (Keep VERY thin)

Contains:

- Entities

```
public class Candidate
{
    public string Id { get; set; }
    public List<string> Skills { get; set; }
    public float[] Embedding { get; set; }
}
```

👉 No logic here

🟡 2. Application Layer (MOST IMPORTANT)

Contains:

- Ranking logic
- Skill extraction
- Interfaces

```
public interface IEmbeddingService
{
    Task<float[]> GenerateEmbedding(string text);
}
```

```
public class RankingService
{
    public List<Candidate> Rank(...)
}
```

👉 This is your **core system**

🟣 3. Infrastructure Layer

Contains:

- Cosmos DB client
- Blob storage client
- Queue publisher

- OpenAI integration

```
public class CosmosRepository : ICandidateRepository
```

4. API Layer

- Controllers only
- Calls Application layer

5) How It Fits Your Pipeline

Ingestion Flow

API → Infrastructure (Blob + Queue)

Processing (Function App)

Function → Application (parse + extract + embed)
→ Infrastructure (store in Cosmos)

Matching Flow

API → Application (ranking logic)
→ Infrastructure (fetch from Cosmos)

6) Pros of Using This Approach

✅ 1. Clean Separation

- Easy to understand
- Easy to extend

✅ 2. Future-Proof

When you add:

- Azure Cognitive Search

👉 Only Infrastructure changes

👉 Application (ranking logic) stays same

✅ 3. Testability

You can test:

- Ranking logic
- Skill extraction

Without Azure dependencies

⚠️ 7) Cons (If Overdone)

❌ Too many abstractions

- Interfaces everywhere
- Generic repositories
- Over-engineered layers

👉 Slows you down

❌ Boilerplate code

- More files
- More setup

8) Final Recommendation (Very Clear)

For YOUR MVP:

👉 Use:

✅ "Light Clean Architecture"

- Keep layers minimal
- Focus on:
 - Application layer (ranking logic)
 - Infrastructure (Azure integration)

❌ Avoid:

- Complex domain models
- CQRS
- MediatR (optional, not required)
- Over-abstraction



Final One-Line Answer (Interview Ready)



Clean Architecture is a good fit conceptually, but for MVP I would use a lightweight version to maintain separation of concerns without adding unnecessary complexity.



If You Want Next

I can:

- Generate **actual .NET project structure (folders + files)**
- Or give **sample code for RankingService + CosmosRepository**
- Or help you explain this in an **interview answer format**





Generate actual .NET project structure (folders + files)



Here's a **practical .NET project structure** using a **lightweight Clean Architecture**, tailored for your MVP (Cosmos DB + Queue + Function + Semantic Ranking).

This is **simple enough to build in a few hours**, yet clean and extensible.

Solution Structure

```
TalentIntel.sln
|
├─ TalentIntel.API
├─ TalentIntel.Application
├─ TalentIntel.Domain
├─ TalentIntel.Infrastructure
├─ TalentIntel.Functions
└─ TalentIntel.Tests (optional)
```

1) TalentIntel.Domain (Core Models)

👉 Keep this **very thin**

```
TalentIntel.Domain
|
├─ Entities
|   └─ Candidate.cs
|   └─ Job.cs
|
└─ Enums (optional)
```

Example: `Candidate.cs`

```
public class Candidate
{
    public string Id { get; set; }
    public string Name { get; set; }
    public List<string> Skills { get; set; }
    public float[] Embedding { get; set; }
    public int ExperienceYears { get; set; }
    public DateTime UpdatedAt { get; set; }
}
```



2) TalentIntel.Application (Core Logic)

👉 This is the **heart of your system**

```
TalentIntel.Application
|
├── Interfaces
|   ├── ICandidateRepository.cs
|   ├── IEmbeddingService.cs
|   ├── IBlobService.cs
|   └── IQueueService.cs
|
├── Services
|   ├── RankingService.cs
|   ├── SkillExtractionService.cs
|   └── ParsingService.cs
|
└── Models
    └── RankingResult.cs
```

Example: `ICandidateRepository.cs`



```
public interface ICandidateRepository
{
    Task<List<Candidate>> GetCandidatesBySkillsAsync(List<string> skills);
    Task UpsertCandidateAsync(Candidate candidate);
}
```

Example: RankingService.cs

```
public class RankingService
{
    public List<(Candidate candidate, double score)> Rank(
        float[] jobEmbedding,
        List<Candidate> candidates)
    {
        return candidates
            .Select(c => (c, CosineSimilarity(jobEmbedding, c.Embedding)))
            .OrderByDescending(x => x.Item2)
            .Take(10)
            .ToList();
    }

    private double CosineSimilarity(float[] a, float[] b)
    {
        double dot = 0, magA = 0, magB = 0;

        for (int i = 0; i < a.Length; i++)
        {
            dot += a[i] * b[i];
            magA += a[i] * a[i];
            magB += b[i] * b[i];
        }

        return dot / (Math.Sqrt(magA) * Math.Sqrt(magB));
    }
}
```

Example: SkillExtractionService.cs

```
public class SkillExtractionService
{
    private static readonly List<string> Skills = new()
    {
        "c#", ".net", "azure", "sql", "react", "java", "python"
    };

    public List<string> Extract(string text)
    {
        text = text.ToLower();

        return Skills
            .Where(skill => text.Contains(skill))
            .ToList();
    }
}
```

3) TalentIntel.Infrastructure (Azure Integrations)

```
TalentIntel.Infrastructure
|
├── Repositories
|   └── CosmosCandidateRepository.cs
|
├── Services
|   ├── BlobService.cs
|   ├── QueueService.cs
|   └── OpenAIEmbeddingService.cs
|
└── Configurations
    └── CosmosDbSettings.cs
```

Example: CosmosCandidateRepository.cs

```
public class CosmosCandidateRepository : ICandidateRepository
{
    private readonly Container _container;

    public CosmosCandidateRepository(CosmosClient client)
    {
        _container = client.GetContainer("db", "candidates");
    }

    public async Task<List<Candidate>> GetCandidatesBySkillsAsync(List<string>
skills)
    {
        var query = new QueryDefinition(
            "SELECT * FROM c WHERE ARRAY_CONTAINS(@skills, c.skills)")
            .WithParameter("@skills", skills);

        var iterator = _container.GetItemQueryIterator<Candidate>(query);

        var results = new List<Candidate>();

        while (iterator.HasMoreResults)
        {
            var response = await iterator.ReadNextAsync();
            results.AddRange(response);
        }

        return results;
    }

    public async Task UpsertCandidateAsync(Candidate candidate)
    {
        await _container.UpsertItemAsync(candidate);
    }
}
```

Example: OpenAIEmbeddingService.cs

```

public class OpenAIEmbeddingService : IEmbeddingService
{
    public async Task<float[]> GenerateEmbedding(string text)
    {
        // Call Azure OpenAI API
        // Return vector
        return new float[1536]; // placeholder
    }
}

```

4) TalentIntel.API (Entry Point)

```

TalentIntel.API
|
├── Controllers
|   ├── UploadController.cs
|   └── MatchingController.cs
|
├── DTOs
|   ├── UploadRequest.cs
|   └── RankingResponse.cs
|
└── Program.cs

```

Example: UploadController.cs

```

[ApiController]
[Route("api/upload")]
public class UploadController : ControllerBase
{
    private readonly IBlobService _blobService;
    private readonly IQueueService _queueService;
}

```

```

[HttpPost]
public async Task<IActionResult> Upload(IFormFile file)
{
    var url = await _blobService.UploadAsync(file);

    await _queueService.PublishAsync(new
    {
        blobUrl = url,
        type = "resume"
    });

    return Ok();
}
}

```

Example: MatchingController.cs

```

[ApiController]
[Route("api/match")]
public class MatchingController : ControllerBase
{
    private readonly ICandidateRepository _repo;
    private readonly RankingService _rankingService;
    private readonly IEmbeddingService _embeddingService;

    [HttpPost]
    public async Task<IActionResult> GetTopCandidates([FromBody] string
jobText)
    {
        var jobEmbedding = await _embeddingService.GenerateEmbedding(jobText);

        var candidates = await _repo.GetCandidatesBySkillsAsync(new
List<string> { "azure" });

        var ranked = _rankingService.Rank(jobEmbedding, candidates);

        return Ok(ranked);
    }
}

```



```
}  
}
```

⚡ 5) TalentIntel.Functions (Processing Pipeline)

TalentIntel.Functions
|
├─ ProcessDocumentFunction.cs
└─ Services (optional reuse from Application)

Example: ProcessDocumentFunction.cs

```
public class ProcessDocumentFunction  
{  
    private readonly ParsingService _parser;  
    private readonly SkillExtractionService _skillService;  
    private readonly IEmbeddingService _embeddingService;  
    private readonly ICandidateRepository _repo;  
  
    [Function("ProcessDocument")]  
    public async Task Run([QueueTrigger("documents")] string message)  
    {  
        var text = _parser.Parse(message);  
  
        var skills = _skillService.Extract(text);  
  
        var embedding = await _embeddingService.GenerateEmbedding(text);  
  
        var candidate = new Candidate  
        {  
            Id = Guid.NewGuid().ToString(),  
            Skills = skills,  
            Embedding = embedding  
        };  
    }  
}
```

```
        await _repo.UpsertCandidateAsync(candidate);  
    }  
}
```

6) TalentIntel.Tests (Optional)

```
TalentIntel.Tests  
|  
├── RankingServiceTests.cs  
└── SkillExtractionTests.cs
```

Dependency Flow (Important)

```
API → Application → Domain  
      ↓  
Infrastructure
```

- 👉 Domain has **no dependency**
- 👉 Application depends only on **interfaces**
- 👉 Infrastructure implements them



Final Summary

This structure gives you:

- ✅ Clean separation (without overengineering)
- ✅ Easy Azure integration
- ✅ Testable core logic
- ✅ Future-ready (Cognitive Search plug-in later)
- ✅ Achievable in ~4 hours

If you want next, I can:



- Generate a **GitHub-ready repo with all files** in the above Design.md for a Talent Intelligence & Ranking Engine (MVP) built on .NET + Azure. recreate it with the
- Add **DI setup in Program.cs** following specific changes while keeping all other sections intact and properly formatted in Markdown.
- Or provide **Postman collection**

section 5: Cosmos DB

Update to mention 2 containers:

- Candidates container
- Jobs container

Section 6: Matching Service AND Section 3B: Matching Pipeline

Update both to reflect the new flow:

1. Receive Job ID
2. Fetch the skills and embedding from the 'Jobs' container based on Job ID
3. Fetch filtered candidates from the 'Candidates' container
4. Compute cosine similarity
5. Rank and return Top 10

Section 10: Future Improvements

Replace the existing list with the following items (keep any non-conflicting originals if they still apply):

- Networking implementation to Azure Resources (VNet integration, private endpoints)
- Use of Log Analytics Workspace with DCR (Data Collection Rules) and DCE (Data Collection Endpoints)
- Use of Managed Identities instead of connection string keys
- Use of Azure Key Vault for secrets and configuration management
- Authentication and Authorization (e.g., Azure AD / Entra ID)
- Semantics based on multiple factors instead of just skills (e.g., experience, seniority, role context, domain)

- Integrate Azure Cognitive Search for vector search
- Replace keyword extraction with NLP/LLM-based parsing
- Introduce ML-based ranking (learning-to-rank)
- Add feedback loop (clicks, hires)
- Add caching layer (Redis)
- Support multi-tenancy and access isolation
- Implement compliance features (data privacy, deletion)

Create the Actual Architecture with the name of the Projects shown in the Project Structure.

recreate the full document with these changes applied, using clean Markdown formatting with proper headings, tables, code blocks, and bullet points throughout.



Here is your **updated, submission-ready** DESIGN.md with all requested changes applied and properly structured.

DESIGN.md

Talent Intelligence & Ranking Engine (MVP)

1. Overview

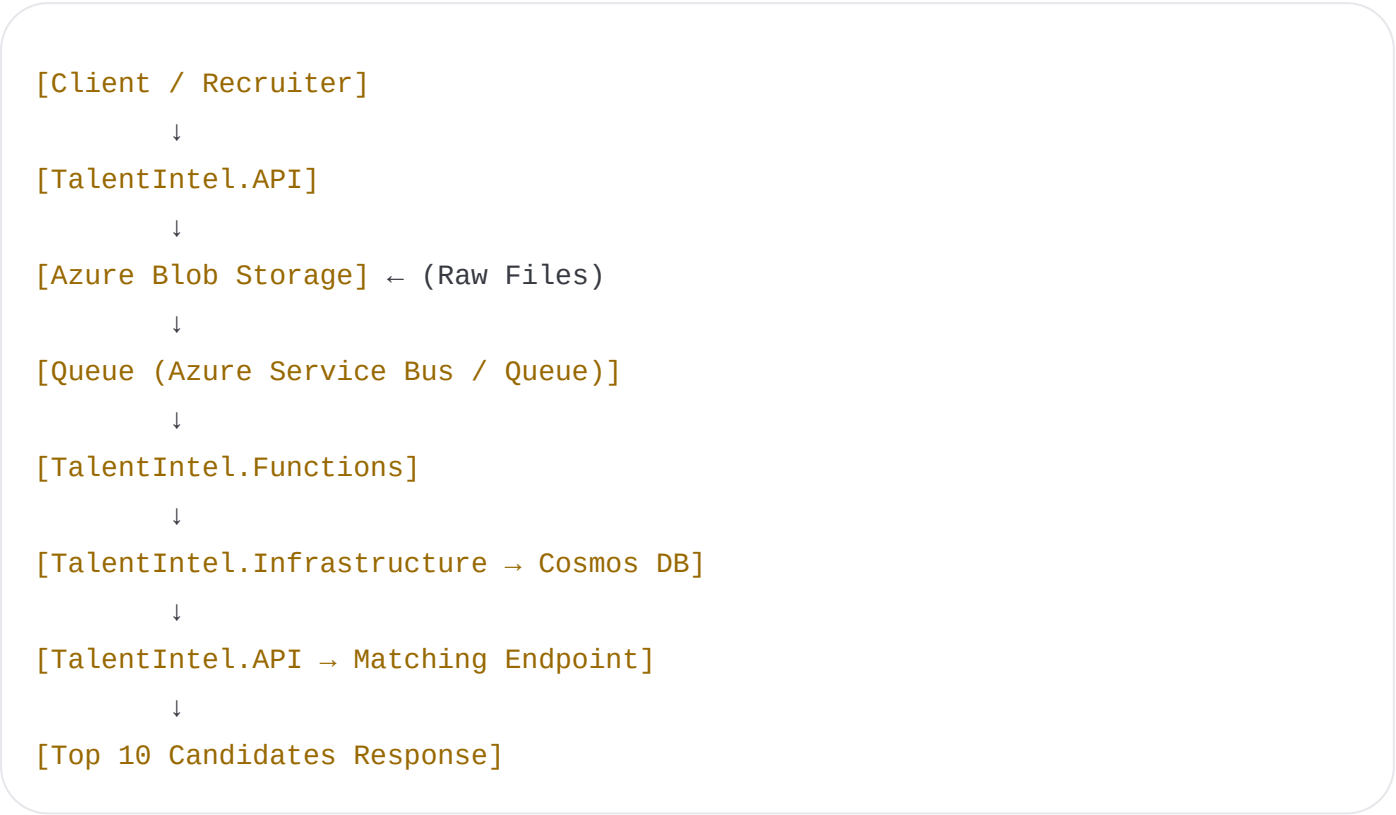
This system replaces a legacy keyword-based recruitment engine with a **semantic, AI-driven matching system** that ranks candidates against job descriptions.

The MVP focuses on:

- End-to-end pipeline (ingestion → parsing → embedding → ranking)
- Semantic similarity using embeddings
- Scalable architecture with minimal complexity
- Cloud-native implementation using **.NET + Azure**

2. Architecture

High-Level Architecture (Project-Based)



Project Mapping

Project	Responsibility
TalentIntel.API	API endpoints (upload, match)
TalentIntel.Application	Core business logic (ranking, parsing, skills)
TalentIntel.Domain	Entities/models
TalentIntel.Infrastructure	Azure integrations (Cosmos, Blob, Queue, OpenAI)
TalentIntel.Functions	Background processing (parsing, embedding)

Components

1. API Layer (ASP.NET Core)

- Upload resumes and job descriptions

- Fetch Top 10 candidates
 - Stateless, horizontally scalable
-

2. Azure Blob Storage

- Stores raw documents (PDF, DOCX)
 - Acts as durable file storage
-

3. Queue (Azure Queue / Service Bus)

- Decouples ingestion from processing
 - Enables retry and reliability
-

4. Azure Function (Processing Worker)

Handles:

- Document parsing
 - Skill extraction (lightweight)
 - Embedding generation (via Azure OpenAI)
-

5. Cosmos DB

The system uses **two containers**:

- **Candidates container**
 - Stores candidate profiles
 - Skills, embeddings, metadata
 - **Jobs container**
 - Stores job descriptions
 - Extracted skills and embeddings
-

6. Matching Service (.NET API)

- Receives Job ID
 - Fetches job data from Cosmos DB
 - Fetches filtered candidates
 - Computes similarity (cosine similarity)
 - Ranks and returns Top 10
-

3. Data Flow

A. Ingestion Pipeline

1. User uploads Resume / JD
2. API stores file in Blob Storage
3. API publishes message to Queue
4. Function processes message:
 - Parse document → extract text
 - Extract skills (keyword-based)
 - Generate embedding
5. Store structured data in Cosmos DB
 - Candidates → Candidates container
 - Jobs → Jobs container

B. Matching Pipeline

1. Receive Job ID
 2. Fetch job (skills + embedding) from Jobs container
 3. Query Candidates container (filtered by skills/metadata)
 4. Compute cosine similarity in application layer
 5. Rank candidates
 6. Return Top 10
-

C. Data Model (Cosmos DB)

Candidate Document

```
{
  "id": "candidate-123",
  "type": "candidate",
  "name": "John Doe",
  "skills": ["c#", "azure", "sql"],
  "experienceYears": 5,
  "embedding": [0.21, 0.77, ...],
  "updatedAt": "2026-04-05",
  "embeddingVersion": "v1"
}
```

Job Document

```
{
  "id": "job-456",
  "type": "job",
  "title": "Senior .NET Developer",
  "skills": ["c#", ".net", "azure"],
  "embedding": [0.45, 0.12, ...],
  "createdAt": "2026-04-05",
  "embeddingVersion": "v1"
}
```

4. Ranking Logic

Scoring Function

```
Score =
  0.8 * semantic_similarity +
  0.2 * skill_overlap
```

Matching Steps

1. Retrieve job embedding
 2. Filter candidates
 3. Compute cosine similarity
 4. Rank and return Top 10
-

5. Scaling Approach

A. Horizontal Scaling

- API and Function Apps are stateless → scale out easily
 - Azure handles auto-scaling
-

B. Asynchronous Processing

- Queue decouples ingestion from processing
 - Prevents API bottlenecks
-

C. Efficient Querying

- Use Cosmos DB filtering:
 - Skills
 - Experience
 - Reduce dataset before similarity computation
-

D. Controlled Candidate Set

- Limit candidates for ranking:
 - Fetch top 500–1000 candidates only
-

E. Parallel Processing

- Use parallel computation for similarity scoring
-

F. Future Scaling Path

- Introduce vector search engine (Azure Cognitive Search)
 - Move similarity computation out of application layer
-

6. Failure Modes & Mitigations

Failure Mode	Impact	Mitigation
Parsing failure	Missing data	Store raw text, retry parsing
Queue message loss	Data inconsistency	Use retry + dead-letter queue
Function failure	Processing delay	Automatic retries
Embedding API failure	Missing embeddings	Retry with exponential backoff
Duplicate processing	Data duplication	Use idempotent upsert in Cosmos DB
High latency (large dataset)	Slow ranking	Pre-filter candidates
High RU consumption	Increased cost	Optimize queries + projections
Cold start (Function App)	Delayed processing	Use warm instances / premium plan

7. Design Decisions

- **Semantic-first ranking** using embeddings
- **Cosmos DB as unified storage**
- Separate containers for:
 - Candidates
 - Jobs
- **Queue-based async pipeline** for ingestion
- **Application-layer ranking** to simplify MVP
- **Skill extraction as supporting signal (not primary)**

8. Trade-offs

Decision	Trade-off
No vector DB	Simpler MVP, but less efficient at scale
App-based similarity	More control, but higher compute cost
Keyword skill extraction	Fast, but less accurate
Cosmos DB only	Fewer components, but weaker search capabilities

9. Limitations

- No native vector indexing (manual similarity computation)
 - Limited NLP capabilities (basic skill extraction)
 - Potential filtering bias (missing relevant candidates)
 - No learning/feedback loop
 - Latency increases as dataset grows
 - Cosmos DB RU cost can rise with inefficient queries
-

10. Future Improvements

- Networking implementation to Azure resources (VNet integration, private endpoints)
- Use of Log Analytics Workspace with DCR (Data Collection Rules) and DCE (Data Collection Endpoints)
- Use of Managed Identities instead of connection string keys
- Use of Azure Key Vault for secrets and configuration management
- Authentication and Authorization (Azure AD / Entra ID)
- Semantic ranking based on multiple factors (experience, seniority, role context, domain)
- Integrate Azure Cognitive Search for vector search
- Replace keyword extraction with NLP/LLM-based parsing
- Introduce ML-based ranking (learning-to-rank)
- Add feedback loop (clicks, hires)

- Add caching layer (Redis)
 - Support multi-tenancy and access isolation
 - Implement compliance features (data privacy, deletion)
-

11. Conclusion

This MVP delivers a **semantic, scalable candidate ranking system** with:

- Clean separation of ingestion and processing
- Efficient filtering + ranking pipeline
- Minimal infrastructure complexity
- Clear upgrade path to a vector-native architecture

It balances **speed of implementation (4-hour constraint)** with **production-aligned design principles**.
